Secure Presence Verification System

Title page

A two-layer attendance verification platform for educational institutions

College / University: **[Enter institution name]**

Department: **[Enter department]**

Submitted by: **[Enter student name(s)]**

Roll number(s): **[Enter roll number(s)]**

Project supervisor: **[Enter supervisor name and designation]**

Academic year / semester: **[Enter academic year and semester]**

Project abbreviation: **SPVS**. Source repository: Secure-Presence-Verification-System-SVPS---Attendance-Software. The inherited repository name uses SVPS; the implemented product and API use SPVS consistently.

Abstract

SPVS is an integrated student attendance system that combines biometric identity evidence with physical-presence evidence. The backend validates a randomized temporal face challenge, a real pretrained face embedding, a device-held signing credential, current GPS measurements and available Wi-Fi observations. It produces explicit present, rejected, review, retry or expired decisions. PostgreSQL transactions and unique constraints prevent duplicate successful attendance, while faculty corrections require attributable reasons.

The implementation uses FastAPI, SQLAlchemy, PostgreSQL, OpenCV SFace/YuNet, MediaPipe, a React institutional dashboard and an Expo student application. It includes actual API integration, migrations, automated tests, model checksums, dependency locks and submission artifacts. Verification evidence is separated from claims that require a physical phone or external attestation configuration. The system demonstrates layered resistance and accountable review without claiming to prevent every spoofing or collusion attack.

1. Introduction

Attendance is both an academic record and a security-sensitive institutional workflow. A presence claim affects eligibility, evaluation and student disputes. A simple token or register makes recording convenient but may provide little evidence about who submitted the claim or where that person was located. At the same time, a complex verification system can harm usability if it silently rejects students or overstates the reliability of noisy sensor data.

SPVS approaches the problem through two primary layers. Identity verification evaluates a consented face template, a fresh challenge and an authorized device. Physical presence verification evaluates location quality, the classroom geofence and supported Wi-Fi signals. Supporting authentication, academic configuration, reports and auditing make those layers usable in an actual college workflow.

2. Problem statement

Traditional methods allow shared tokens, proxy responses, remote submissions and unaccountable edits. A mobile client is an untrusted input source; it must not declare its own face similarity or distance from a classroom. Shared tokens, expired sessions and repeated requests require server-side controls.

Software-only verification cannot absolutely prove presence against a fully compromised phone or coordinated attackers. The engineering problem is therefore to enforce meaningful gates, expose uncertainty, preserve accountability and evaluate the limitations honestly. SPVS excludes surveillance cameras and additional beacon hardware, keeping the project reproducible with normal student phones.

3. Objectives

1. Implement a complete role-based academic attendance workflow.
2. Use actual model-generated face embeddings with consent and protected template storage.
3. Validate randomized temporal challenges on the server.
4. Bind evidence to an authorized device credential and a fresh session.
5. Calculate geofence and signal decisions from submitted measurements on the server.
6. Make retries, replay, concurrency and corrections deterministic and testable.
7. Provide understandable student outcomes and an accountable faculty review process.

8. Deliver source, documentation, measured test evidence and reproducible setup.

4. Existing systems and limitations

Manual registers are simple but allow proxy marks and require transcription. Static QR or code-based systems are convenient but shared tokens can travel beyond the classroom. Pure GPS systems are affected by indoor uncertainty and spoofing. Face-only systems may accept photograph/video presentation if identity matching is confused with liveness. Device identifiers can be copied and do not establish cryptographic possession.

Adding more signals does not automatically establish strong assurance. A high weighted score can conceal a failed security gate if the decision algorithm is poorly designed. SPVS instead enforces mandatory identity/session/device conditions before evaluating supporting location evidence, then routes ambiguity to review or retry.

5. Proposed system

The student receives an institution-managed account, registers one device and consents to biometric enrollment. Faculty open a session for a course and configured classroom. The student requests a bound challenge, performs a short face sequence and collects real foreground location and supported Wi-Fi data. The app signs exact evidence bytes and submits them to FastAPI.

The server checks access, enrollment, session timing, device signature and challenge ownership. It detects and aligns the face, calculates SFace embeddings, compares each frame and analyzes temporal landmarks. Only after these gates pass does it evaluate geofence uncertainty, freshness, Wi-Fi policy and anomalies. A successful decision inserts one attendance record; other states remain attempts that can be disputed and reviewed.

6. Related work and implementation references

The system builds on documented OpenCV face detection/recognition APIs, MediaPipe facial landmarks, modern password hashing and relational transaction controls. OpenCV's published face tutorial supplies an example matching baseline, not an experimental result from this project. Android documentation explains why Wi-Fi scanning and location permissions cannot be assumed to work uniformly. Google and Apple attestation guides describe provider verification boundaries that require external configuration.

The bibliography lists official documentation, pinned model source and actual test-image provenance. No invented publication, benchmark accuracy, institutional deployment or user study is claimed.

7. System requirements

The backend requires Python 3.11/3.12-compatible dependencies, PostgreSQL 15+ (16 in Compose), the pinned model files and generated JWT/Fernet secrets. The dashboard requires a supported Node runtime and a modern browser. Student verification requires a camera-equipped Android/iOS phone, foreground location permission and a reachable API. Android native testing needs a development build, compatible JDK/SDK and the included local Wi-Fi module. iOS requires macOS/Xcode signing and exposes no general Wi-Fi scan API in this implementation.

The primary developer target is a MacBook Air M4. Native ARM64 Python/Node and Docker Desktop are documented. Automated execution in this submission occurred on Linux x86_64; that does not establish Apple Silicon runtime testing.

8. Architecture

The architecture has client, API/service and persistence tiers. The dashboard performs staff operations through a same-origin API proxy. The student app holds credentials in OS secure storage and sends signed evidence. FastAPI enforces roles and scopes, coordinates verification services and records outcomes in PostgreSQL.

The architectural, sequence and decision diagrams in `SYSTEM_ARCHITECTURE.md` are part of this report's generated PDF appendices. OpenCV and MediaPipe inference are synchronous; FastAPI synchronous handlers use its worker thread pool. A per-worker lock guards model instances. Database rate limits and transactional locks remain shared across API workers.

9. Database design

Twenty normalized application tables cover accounts/subtypes, courses/enrollment, classrooms/Wi-Fi, policy, devices/resets, templates, sessions/challenges, attempts/records, events/corrections/disputes, refresh tokens and rate buckets. Foreign keys preserve ownership and attribution. Unique student/session records prevent duplicate official attendance. A partial unique active-device index enforces the normal one-device limit.

Attempts retain reasons and a coarse evidence summary rather than raw images. Corrections retain the actor, reason and previous/new status. Refresh tokens are hashed and grouped into revocable families. The explicit Alembic migration and ER diagram are included in the database appendix.

10. Implementation methodology

Development proceeded from relational/authentication foundations to academic workflows, identity/presence services, client integration, tests and documentation. Each API operation uses a Pydantic input contract. Unknown fields, invalid coordinates and malformed timestamps are rejected. Role-scoped queries and object checks protect backend data independently of UI navigation.

The dashboard provides labeled forms, searchable tables, status labels, real aggregate charts and scoped exports. The mobile app provides enrollment consent, permission handling, sequential camera capture, location acquisition, signing and result/history screens. The Android module reads recent real scan results and reports unavailable evidence when the platform cannot provide it.

11. Identity verification design

YuNet detects exactly one usable face; SFace generates a normalized 128-dimensional embedding from the aligned crop. The enrolled template is encrypted under a key separate from JWT signing. Every frame must match the reference at the configured threshold. Enrollment frames also match one another before averaging.

MediaPipe supplies facial landmarks. A blink requires open/closed/open eye-aspect transitions. Head actions use normalized nose displacement and stable consecutive observations. Frames must be ordered, within the challenge period and separated by plausible time intervals. Challenge IDs are unpredictable and single-use. These controls provide basic replay resistance, not certified capture provenance or comprehensive deepfake resistance.

12. Physical presence design

Haversine distance is calculated on the server. Location age, future timestamps, uncertainty and reported mock indicators are checked before accepting a geofence. An uncertainty circle fully outside the radius rejects; an overlapping boundary routes to review. Android Wi-Fi observations are matched by known BSSID and minimum RSSI, requiring at least half of configured references when available.

An explicit policy controls missing Wi-Fi: review by default, reject if mandatory, or GPS-only acceptance at reduced assurance. A coarse travel rule flags implausible movement over short periods. Unconfigured platform attestation never returns a verified result. Numerical risk communicates concern and never bypasses a failed mandatory gate.

13. Security design

Passwords use Argon2id. JWTs validate issuer, audience, expiry and token type. Refresh sessions rotate transactionally and revoke their family after reuse. Device HMACs cover both an idempotency key and exact JSON bytes, using a random credential stored securely on the phone and encrypted in the database.

Student/challenge/session locks protect the attendance transaction. A retry returns the existing attempt only when the signed evidence matches. A changed payload conflicts, and a consumed challenge cannot be reused with a new key. A faculty correction uses the same session lock and produces a separate audit trail. Input body/decoded-image bounds and shared rate buckets constrain abuse.

14. Testing and validation

The project includes unit, database integration, client interaction, browser and real-model smoke tests. PostgreSQL-backed tests run the migration and verify role restrictions, CRUD, refresh replay, device mismatch/revocation, consent, session expiry, duplicates and concurrent retries. Rule tests cover location and temporal sequences. Browser tests interact with the actual running API and database. Mobile logic tests check signing interoperability and decision wording; Metro/Hermes export checks bundle construction.

The API happy-path tests explicitly replace vision to isolate transaction behavior. Real-model tests separately compare public portraits and reject zero/multiple faces. Neither proves full live-phone verification. Current commands, counts and outcomes are included from `TEST_RESULTS.md` in the generated report. Failed intermediate runs were investigated and are not represented as passing results.

15. Actual results

The source contains functional backend, dashboard and student app implementations rather than generated placeholders. Authentication, academic configuration, sessions, reports and audits have been exercised against PostgreSQL. Real pretrained face-model inference has been executed. Verification state handling is connected to the API and constrained database writes. The dashboard builds and browser workflows connect to actual endpoints; mobile logic and bundle checks are recorded separately.

The final result table is generated from executed test result files. No population biometric accuracy, real-device liveness success rate, production latency, enrollment count or deployment adoption is reported because those studies were not performed. Native build status, remaining upstream dependency advisories and provider limitations appear in the verification appendix.

16. Limitations

The system cannot guarantee presence against compromised clients or coordinated credential sharing. Basic landmark liveness can be attacked, GPS has indoor uncertainty, Wi-Fi is platform-dependent and attestation is unconfigured. Thresholds need institution-specific calibration and a consented evaluation dataset. Access tokens have an expiry window after logout; audit storage is not externally immutable. Large-class performance, retention automation, key rotation and fully managed production deployment require further engineering.

The current Expo dependency tree has unresolved upstream advisories recorded in the limitations appendix. These must remain visible in a college evaluation and future release decision. Real-device and Apple Silicon checks remain separate from cloud test results.

17. Future enhancements

Priorities are provider-backed integrity validation, stronger capture provenance/presentation-attack detection, accessible alternative attendance workflows, calibrated thresholds, measured false-accept/false-reject behavior, pagination, operational retention and protected external audit export. Future work should be based on observed weaknesses and institutional requirements. Excluded beacon, ultrasound, surveillance and blockchain hardware/systems are not proposed as hidden dependencies of this implementation.

18. Conclusion

SPVS demonstrates how identity and location evidence can be combined with server-side enforcement and attributable review in an actual attendance application. The strongest implemented properties are explicit decision gates, relational consistency, evidence-bound retries and accountable corrections. Its practical value depends on honest handling of uncertainty and measured device behavior rather than an absolute anti-proxy claim.

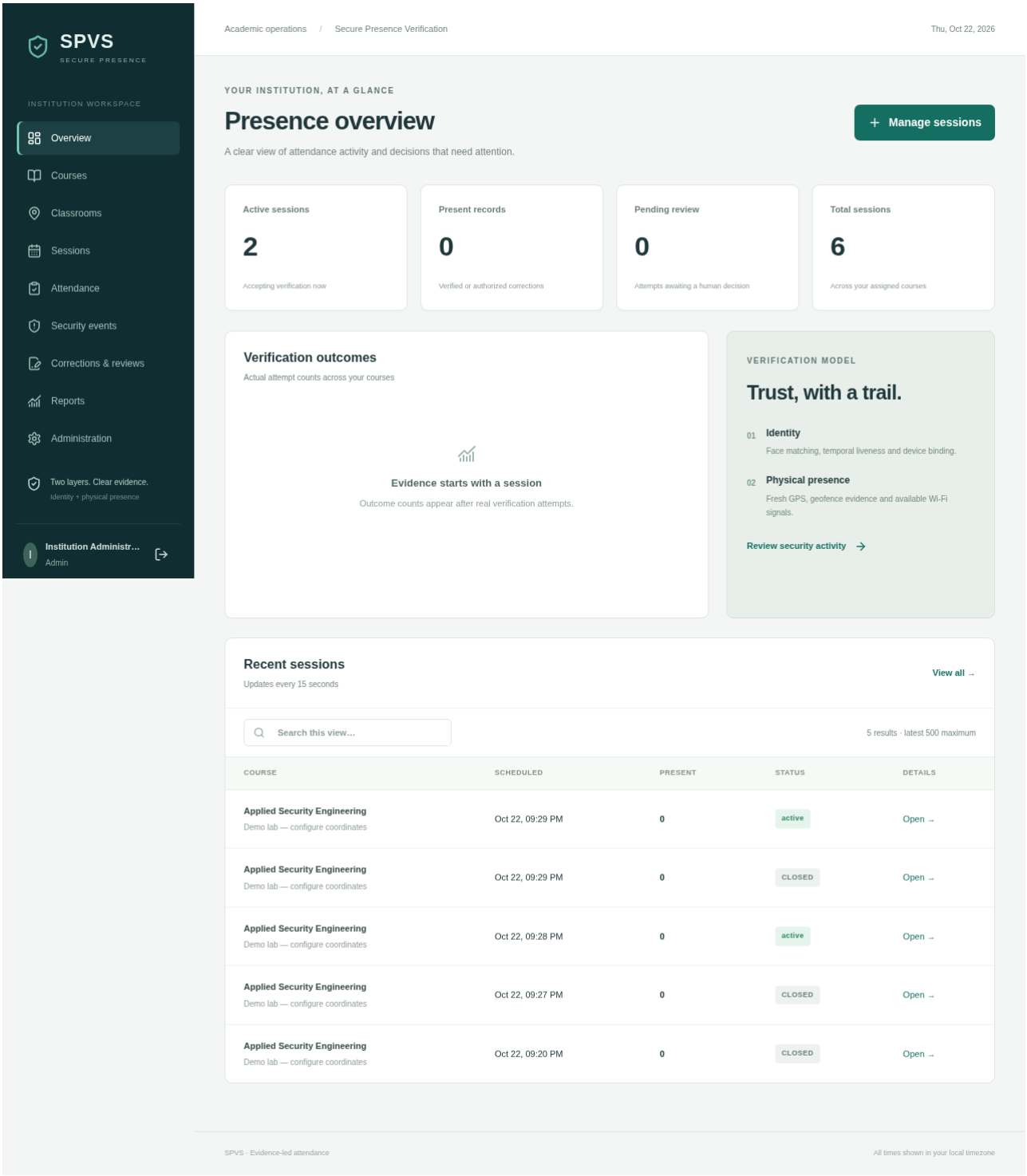
19. References

The full bibliography is in `REFERENCES.md` and is included in the report appendix. It contains official framework, model, operating-system and security documentation plus the actual public test-image sources and relevant unresolved advisories.

20. Appendices

The generated PDF appends architecture/ER/sequence diagrams, the API map, biometric and security design, threat model, current test results, requirements traceability and known limitations. Editable source Markdown and the packaging script are included in the ZIP so that the student can add genuine institutional measurements and academic details without rebuilding the software from scratch.

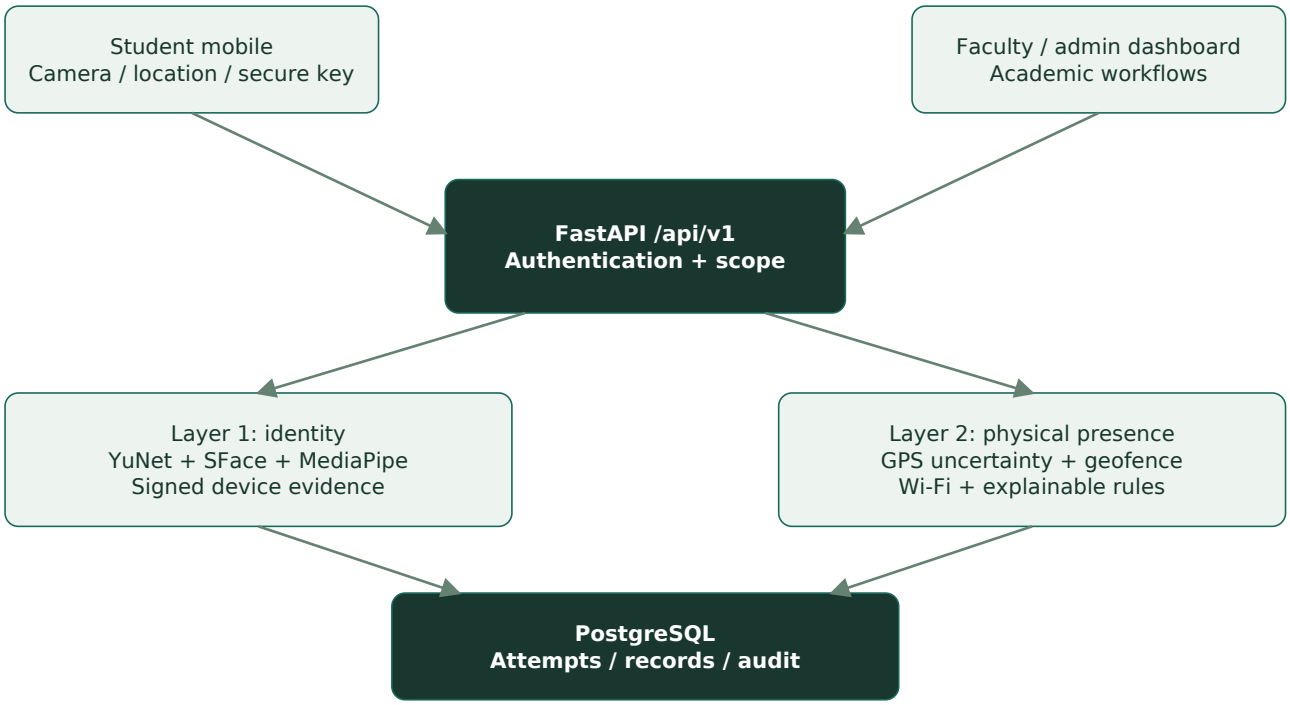
Application evidence · actual dashboard



Actual browser screenshot from the evaluated workspace. Counts reflect real demo activity; no attendance was fabricated.

System architecture

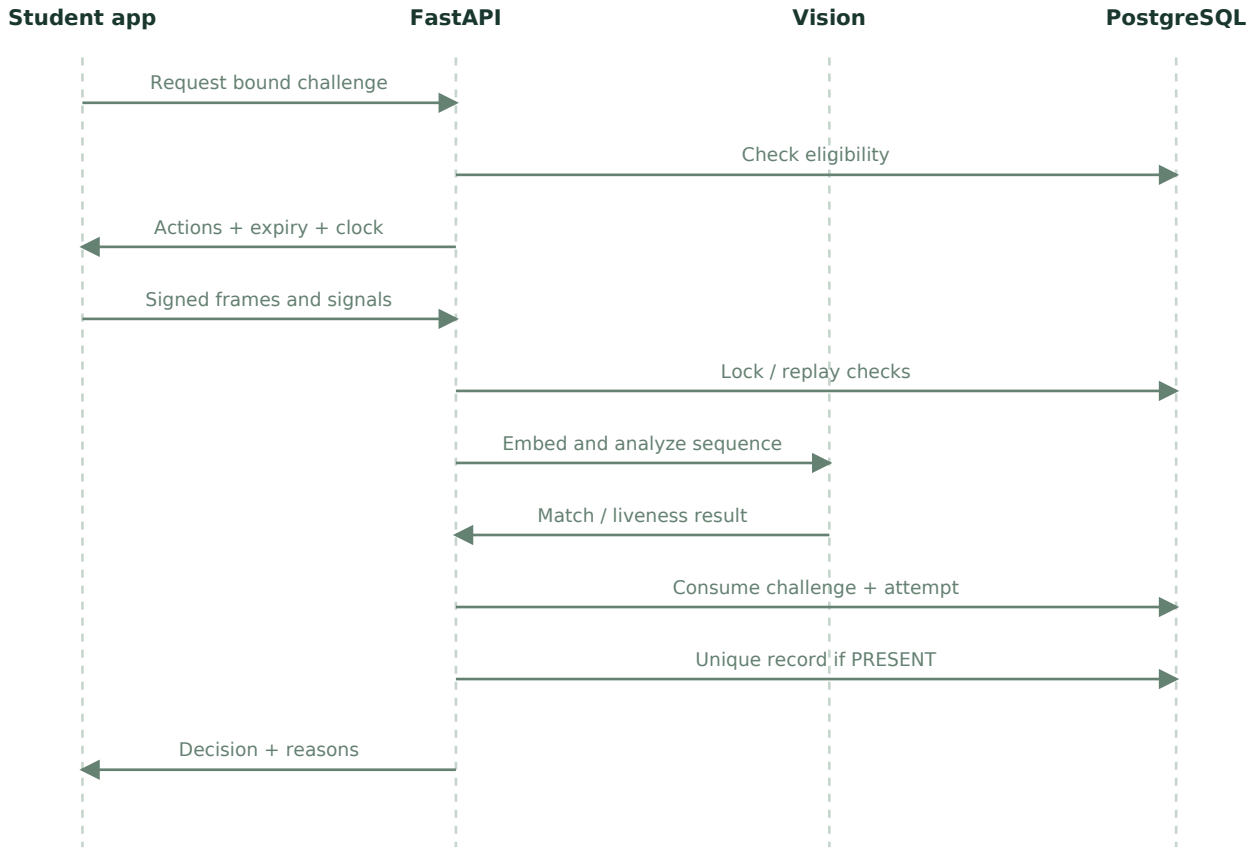
Component architecture



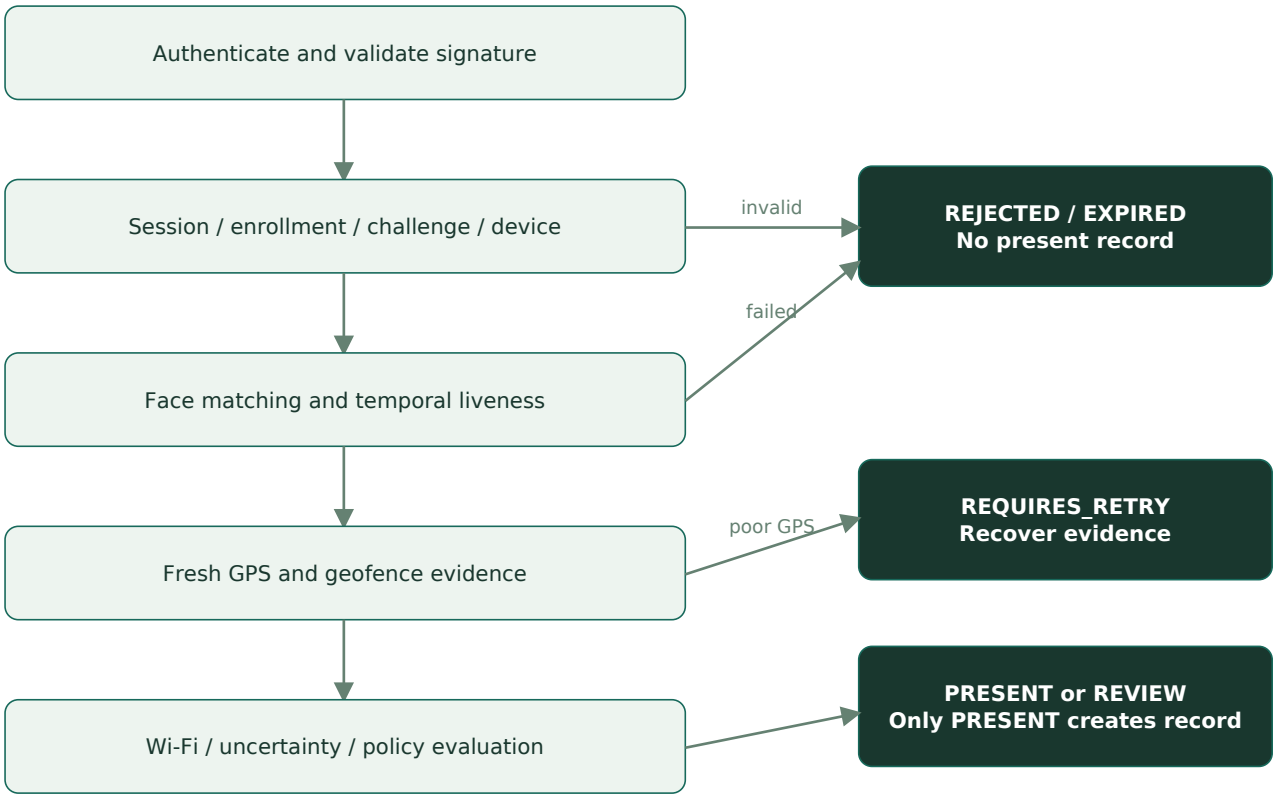
The backend uses synchronous SQLAlchemy sessions in FastAPI synchronous route handlers, which execute in the framework's thread pool. This is deliberate: OpenCV/MediaPipe inference is synchronous and guarded by a process-local model lock. The application does not block an async endpoint with a synchronous database call. Multiple API workers share PostgreSQL rate limits and locks; each worker has its own model memory. Capacity and fair scheduling require load testing before institutional deployment.

The dashboard uses a same-origin `/api` route through Vite in development or nginx in containers. Tokens remain in dashboard memory. The mobile app stores tokens and a per-user device signing credential in SecureStore. Its camera cache files are deleted after conversion; no raw frames are saved by the backend.

Verification sequence



Decision flow

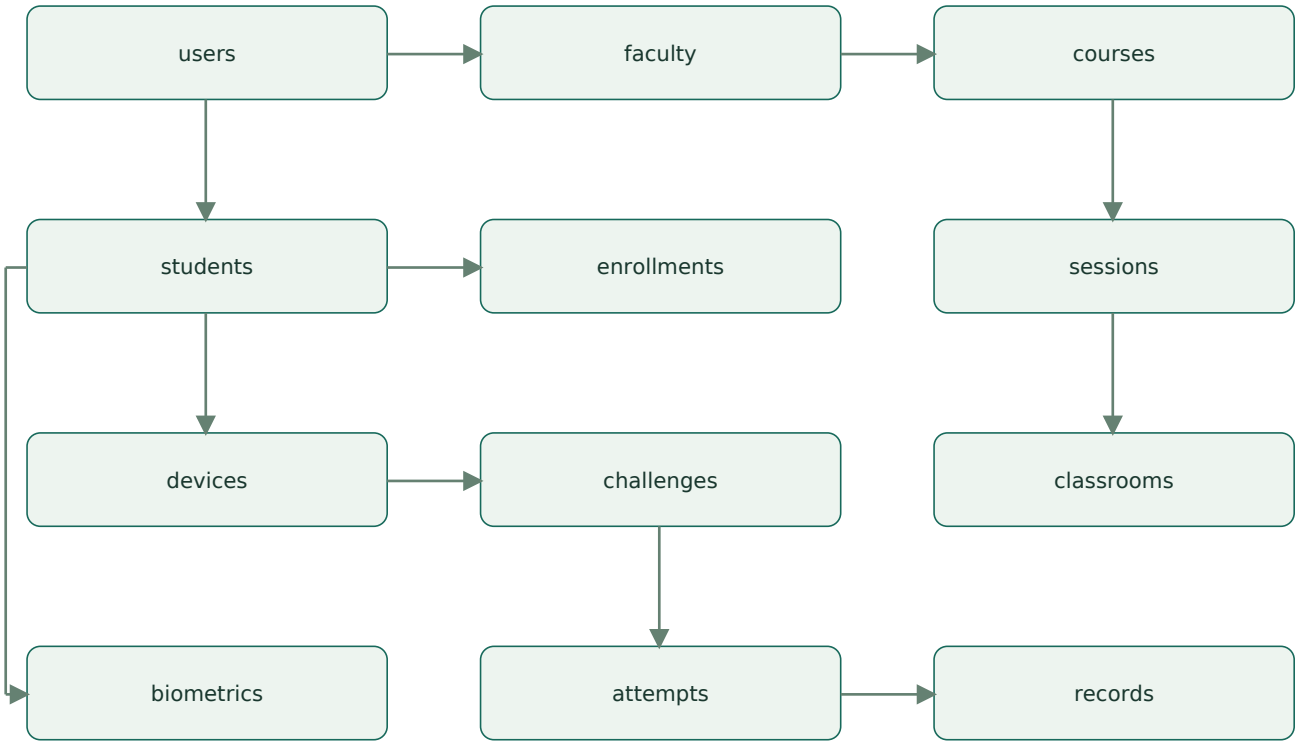


Authentication, malformed payload and signature failures are HTTP errors; signed, attributable verification outcomes are persisted as attempts. This distinction prevents unauthenticated requests from inventing someone else's attempt history.

Database design

PostgreSQL 16 is used in Compose. The initial migration creates 20 application tables plus Alembic's version table. IDs are application-generated UUID strings. Timestamps are timezone-aware. There are no raw-image columns.

Table	Purpose / important constraint
users	Unique email, password hash, current role and active status
students	User FK and unique roll number
faculty	User FK and department
courses	Unique code, assigned faculty FK
enrollments	Composite primary key: course and student
classrooms	Unique name, latitude, longitude, radius
wifi_references	Unique classroom/BSSID and minimum RSSI
policies	Singleton verification settings
devices	Encrypted signing key; partial unique index allows one active device per student
device_resets	Requested reason, status and administrator reviewer
biometric_profiles	Student primary key, encrypted template, model version, consent timestamp
attendance_sessions	Course/classroom FKs, start/end and explicit closure
challenges	Student/device/session bindings, purpose, random actions, expiry and used flag
attendance_attempts	Unique student/idempotency key, payload hash, decision, reasons and evidence summary
attendance_records	Unique student/session, optional attempt FK, status and source
security_events	Actor/session, severity, structured details and review flag
corrections	Record/actor FKs, previous/new status and mandatory reason
disputes	Student/attempt, reason, resolution and reviewer
refresh_tokens	Hashed random token, family, expiry, used and revoked flags
rate_buckets	Shared fixed-window counters keyed by account or connection source



Core foreign-key relationships; full schema and remaining tables listed in the database chapter.

Verification locks the student row, then challenge and session, before consuming the challenge and writing the attempt/record. The same session row serializes a manual correction against verification. Device reset approval locks the student before revocation. A refresh-token rotation locks its token row; reuse revokes the entire refresh family. These transactions complement, rather than replace, unique constraints.

Subtype rows are retained when a current role changes because historical foreign keys must remain valid. Authorization always reads the current user role. Deleting referenced academic entities is intentionally restricted; deactivation and enrollment removal are supported while history remains attributable.

Location summaries retain coordinates rounded to three decimal places for coarse travel anomaly checks, plus distance, accuracy and age. Define an institutional retention schedule before deployment. The current application does not implement automated retention purges or an immutable external audit sink.

API documentation

The authoritative machine-readable contract is `docs/openapi.json`, exported from the implemented application, and the live `/openapi.json`. The versioned API prefix is `/api/v1`. Protected operations use `Authorization: Bearer <access token>`.

Endpoint groups

Group	Main routes	Authorization
Authentication	POST `/auth/login`, `/auth/refresh`, `/auth/logout`; GET `/auth/me`	Public token exchange; authenticated profile
Registration	POST `/auth/register`	Disabled by default; student-only when enabled
Accounts	GET/POST `/users`; PATCH `/users/{id}`	Administrator
Academics	GET/POST `/courses`; GET/POST/DELETE enrollment routes	Course-scoped reads; administrator writes
Classrooms	GET/POST `/classrooms`; PUT `/classrooms/{id}`	Staff read; administrator write
Sessions	GET/POST `/sessions`; GET `/{id}`; POST `/{id}/close`	Enrolled student read; assigned faculty/admin manage
Biometrics	GET/DELETE `/biometrics`; POST `/biometrics/enroll`	Student's own template
Device	GET/POST `/devices`; reset request/list/approval routes	Student ownership; admin reset approval
Evidence	POST `/challenges`, `/attendance/verify`	Authenticated student and active bound device
Review	GET/POST `/disputes`; POST `/{id}/resolve`; GET/POST `/corrections`	Student ownership; scoped staff decisions
Audit	GET `/events`; POST `/{id}/review`	Scoped faculty or administrator
Records	GET `/attempts`, `/records`	Student own data or scoped staff data
Reporting	GET `/overview`, `/reports/attendance.csv`	Scoped staff
Policy	GET/PUT `/policy`	Authenticated read; administrator write with reason

Signing protocol

`POST /attendance/verify` and `/biometrics/enroll` accept three fields:

```
{
  "payload": "<exact serialized Evidence JSON string>",
  "signature": "<64 lowercase hexadecimal HMAC-SHA256 characters>",
  "idempotency_key": "<16 to 64 characters: letters, digits, hyphens or underscores>"
}
```

HMAC uses the UTF-8 device credential string as key and the bytes of `idempotency_key + "." + payload`. The inner `Evidence` schema has challenge/device IDs, 8–40 timestamped JPEG frames, consent, optional location, real Wi-Fi observations/status and optional integrity token. Coordinates, accuracy and timestamp are measurements; no `distance`, `inside_geofence`, `similarity` or `liveness_passed` field is accepted as proof. Clients should keep the same exact signed request and key after a network timeout to recover the original attendance decision.

Enrollment and attendance purposes are not interchangeable. An enrollment challenge cannot reference a session; an attendance challenge must bind an active, enrolled session and an existing biometric profile. Enrollment currently returns a conflict after a successful enrollment replay rather than supporting durable enrollment idempotency; the client can query `/biometrics` to resolve an uncertain enrollment response.

Outcomes and errors

An attributable attendance attempt contains `id`, `state`, `reasons`, `risk_score` and `evidence_summary`. `PRESENT` creates one official record. `PENDING_REVIEW`, `REQUIRES_RETRY`, `REJECTED` and `EXPIRED` do not create a present record. Faculty may create an explicitly sourced correction later.

HTTP errors use `{"error":{"code":403,"message":"..."}}`. Validation responses include field locations without echoing sensitive input. Typical codes are 401 authentication failure, 403 role/device/signature denial, 404 missing resource, 409 conflict/replay, 413 body limit, 422 invalid evidence, 429 rate limit and 503 missing verification service/models. Never treat an HTTP failure as an attendance approval.

Most operational lists expose the latest 500 rows; course and enrollment lists are not paginated. CSV export includes all records in the authorized selection. Large institutions need cursor pagination, query optimization and retention jobs before deployment.

Biometric design

Model provenance and separation of responsibilities

YuNet detects a face and five alignment landmarks. It does not establish identity. SFace generates a 128-dimensional learned face representation, using OpenCV's `FaceRecognizerSF.alignCrop` and `feature` APIs. MediaPipe Face Mesh separately supplies dense facial landmarks for temporal liveness. ONNX models are executed by OpenCV DNN on CPU; ONNX Runtime is unnecessary for this pipeline.

Asset	Provenance	License
`face_detection_yunet_2023mar.onnx`	OpenCV Zoo, commit `48b10fa505aeb51fbd127fc60a5c660040e823b7`	MIT; Shiqi Yu notice retained
`face_recognition_sface_2021dec.onnx`	Same official repository/commit	Apache 2.0; full license retained
`face_landmarker.task` float16 v1	Official Google MediaPipe model storage; consumed by MediaPipe 1.1.1 Tasks	MediaPipe code is Apache 2.0; consult the linked official model card/provider terms for the pretrained asset

Expected SHA-256 values are in `scripts/download_models.py`, copied from the official Git LFS pointer objects. A mismatch stops installation. The SFace asset is 38,696,353 bytes and YuNet is 232,589 bytes. No model is trained in this project.

The landmark task is downloaded over verified HTTPS from the versioned official Google URL. Its SHA-256 is pinned as `64184e229b263107bc2b804c6625db1341ff2bb731874b0bcc2fe6544e0bc9ff`; it is loaded in IMAGE mode with at most two faces. The current MediaPipe Tasks API avoids the legacy package's protobuf dependency.

Enrollment and comparison

Enrollment requires an authenticated student, an authorized device, explicit consent, a fresh enrollment-purpose challenge and a server-validated sequence. JSON cannot carry a trusted embedding or similarity score. JPEG input is base64-decoded, limited to dimensions 160–1280, checked for brightness and Laplacian sharpness, and must contain exactly one usable YuNet face of at least 70 pixels. The aligned BGR crop passes through OpenCV's SFace preprocessing. Returned vectors must contain exactly 128 finite values and have nonzero norm. They are L2-normalized.

Every verification frame is embedded and compared against the encrypted enrolled template. Enrollment frames must match the first frame so that a sequence cannot trivially switch people. The minimum similarity across frames must meet `FACE_THRESHOLD`, default **0.5**. OpenCV's tutorial gives a cosine baseline of approximately 0.363 for its example evaluation; 0.5 here is a stricter initial operating point, not a locally calibrated false-acceptance guarantee. A realistic institution must collect consented evaluation data and measure FAR/FRR across lighting, pose and user groups before choosing a threshold.

The enrollment template is the normalized mean of sequence embeddings and is encrypted with Fernet under a separate `BIOMETRIC_KEY`. Templates carry an explicit model version. Raw frame bytes and unencrypted embeddings are not persisted. Student deletion requires password confirmation. Backup retention can still retain deleted ciphertext; operators must apply their retention policy to backups as well.

Liveness protocol

The server issues an unpredictable UUID challenge bound to a student, device, purpose and attendance session (when applicable). It selects a left/right turn and randomizes the turn/blink ordering, inserting center poses. A challenge normally expires after 60 seconds and is consumed once. Session attempts and challenge issuance are rate limited.

Face Mesh computes eye aspect ratio (EAR) from six landmarks per eye, averaged across both eyes. A blink must transition open (`EAR > 0.23`) → closed (`EAR < 0.19`) → open. Head direction is a signed nose displacement relative to eye centers, normalized by interocular distance. Center is absolute displacement below 0.12; turns exceed 0.18 in the requested direction. Two consecutive observations must support each non-blink pose. This is a heuristic displacement proxy rather than a calibrated three-dimensional head-pose estimator.

A sequence contains 8–40 frames, has increasing timestamps separated by 0.08–2.5 seconds, spans at least two seconds, remains inside the challenge time window, and may not claim a final capture more than two seconds in the future. Identical encoded frames are rejected. Mobile captures unmirrored camera images and synchronizes capture timestamps to the challenge's server clock. Direction conventions, timing and EAR thresholds require physical-camera validation; basic frame rates can miss a fast blink, so the UI asks for a slow close/open action.

Limits and measured evidence

Landmark liveness rejects a static frame and some naive replay attempts. It does not prove camera provenance and can be defeated by sophisticated replay, virtual-camera injection, deepfakes or a compromised client. Randomized action order and signatures increase the effort but are not certified presentation-attack detection. Client timestamps are assertions checked for consistency, not trusted hardware timestamps.

Real model tests use the public NASA astronaut image from scikit-image and Grace Hopper image from matplotlib: same-face photometric variation, different-person comparison, zero faces and multiple faces. These smoke tests validate model operation; four tests on two portraits are not a biometric accuracy study. No live student was enrolled during automated evaluation.

Security architecture

Authentication and authorization

Passwords use Argon2id with memory cost 65,536 KiB, time cost 3 and parallelism 2. Failed logins use a constant-work password check against a dummy hash when the account is absent. Tokens are HS256 JWTs with explicit issuer, audience, type, issued-at, expiry, subject and random ID; the secret must be at least 32 characters and is randomly generated by setup. Access lifetime defaults to 15 minutes. Refresh tokens are random 48-byte URL-safe values stored only as SHA-256 hashes, with a default seven-day lifetime. Rotation is transactional; reused tokens revoke their refresh family.

Logout revokes refresh sessions. Already-issued access tokens remain valid until expiry unless the account is disabled; endpoints check active status and current role on every request. This access-token revocation window is documented, not concealed. Role updates revoke refresh tokens and device bindings and are administrator-only with reasons. The dashboard keeps credentials in memory, so reload requires login. Mobile credentials use OS secure storage.

Device binding

After password confirmation the server issues a random 256-bit credential, returned once. It is encrypted server-side and stored through Expo SecureStore on the phone. A signed request covers the UTF-8 bytes of `idempotency_key + "." + payload`, using HMAC-SHA256. `payload` is the exact JSON string sent to the server, eliminating cross-language JSON canonicalization ambiguity. A static device identifier alone is never accepted as proof.

The database has a partial unique active-device index. Approved reset revokes old devices. This is possession of a device-held secret, not hardware-backed signing attestation. Android SecureStore uses Keystore-backed encryption, and iOS uses Keychain; root/jailbreak extraction, backups, instrumentation and credential sharing remain possible. The included attestation adapter always returns unverified. To add Play Integrity/App Attest, verify provider signatures, app/package identity, signing-certificate digest, nonce binding, expiry, replay and provider verdict before changing that boundary. Never forward a client boolean as a verdict.

Evidence and decisions

Identity gates precede presence scoring. Enrollment, session, face, challenge and device failures cannot be overridden by a good GPS score. Server-owned rules calculate distance and signal match. A fully outside uncertainty circle rejects; an overlapping boundary requests review. Missing/inaccurate/stale data requests retry or follows the explicit Wi-Fi policy. Mock-location flags reject when reported, but the absence of a flag does not prove an honest OS.

The verified request atomically consumes its challenge, stores an attempt and optionally inserts one attendance record. Retries with the same user/key and exact payload return the original attempt; modified payloads conflict. A different key cannot reuse a consumed challenge. Database locks serialize concurrent retries. Manual corrections are distinct records with actor, reason and before/after status.

Storage and operations

Fernet protects templates and signing credentials at rest with a dedicated key. Secrets, caches and generated local state are excluded from archives. There is no raw-image retention endpoint. Location summaries are coarse and restricted by role. Production operators should use separate least-privilege migration/runtime database accounts, secret management, key rotation with re-encryption, a retention process, encrypted backups and trusted TLS termination.

Rate buckets live in PostgreSQL, shared across workers. The API does not trust caller-supplied forwarded headers for rate identity; behind a reverse proxy, connection IP limits can group multiple clients. Account limits remain independent. Configure trusted proxy handling and infrastructure limits for deployment. Bounded request bodies, decoded image dimensions and model serialization reduce resource abuse but do not replace load testing or upstream request throttling.

Audit rows are append-only through ordinary application APIs except their reviewed flag. A database administrator can still alter them: this is not tamper-proof external audit storage. Implement a protected external append-only sink if that is an institutional requirement.

Threat model

Assets include student identity, biometric templates, device credentials, location evidence, attendance integrity, refresh sessions and faculty audit accountability. Trust boundaries are the mobile device to HTTPS API, staff browser to API, API to PostgreSQL, and operator secrets to server runtime. A student client is untrusted even after login.

Threat	Mitigation in this implementation	Residual risk / evaluation
Shared QR/token	No standalone shared token can create attendance; enrolled identity, challenge and device are required	Account and device credential collusion remains possible
Face photo	Actual embedding plus randomized temporal eye/head sequence	Advanced replay/deepfake/injection can mimic landmarks
Client says liveness passed	Schema rejects unknown verdict fields; server analyzes frames	Frame capture provenance is not attested
Hostel or remote attempt	Server GPS geofence, accuracy, age and available Wi-Fi	Compromised phones may forge all signals
GPS spoofing	Mock indicator, stale/accuracy checks, coarse travel anomaly	An omitted/forged mock flag is not trustworthy
Device ID cloning	Random secret and evidence HMAC; one active binding	Extracted secret can be copied; no hardware signing guarantee
Replay or duplicate	Purpose/user/device/session binding, expiry, single use, unique keys and records	Encoded video may be re-rendered to evade byte equality; liveness is basic
Concurrent submission	Student/challenge/session locks and unique constraints	Session-level serialization limits throughput
Faculty accesses another class	Server course-ownership checks and query scoping	Privileged administrator has institution-wide access
Silent attendance edit	Mandatory reasons, correction ledger, structured events	Database-superuser tampering is outside app controls
Refresh-token theft/reuse	Secure storage, hashing, rotation, family revocation	Access tokens have an expiry window; browser XSS can act while logged in
CSV formula injection	Escape dangerous leading formula characters	Downstream spreadsheet handling still needs care
Large payload/decoded image	Actual-byte body bound, JPEG byte and dimensions cap	Valid expensive requests still consume inference capacity
Dependency compromise	Lockfiles, verified model hashes, audit reports	Audits are time-bound and cannot prove absence of defects

Abuse evaluation protocol

Use consented volunteers on real Android devices. Test identical-photo presentation, screen replay, a wrong person, multiple faces, same evidence/key replay, same evidence/new key replay, a second registered device, a revoked device, a distant location, disabled location permission and unavailable Wi-Fi. Inspect both HTTP outcomes and resulting database/audit records. Record model version, threshold, device model, OS version, illumination and capture timing. Never label a human review or an unrun test as successful automatic presence verification.

The public test portraits are used only for model smoke tests. No external biometric service receives student images, and this implementation does not use facial recognition for surveillance or sensitive-trait inference.

Test and build results

Generated from actual result files at **2026-10-23 03:02 IST**. JUnit timestamps reflect the Linux runner clock.

Suite	Passed	Failed/errors	Skipped	Execution platform
Backend / PostgreSQL (includes real-model tests)	72	0	0	Linux x86_64
Dashboard interactions	8	0	0	Linux x86_64
Mobile protocol and UI logic	7	0	0	Linux x86_64
Chromium against running containers	2	0	0	Linux x86_64

Unique suite totals: **89 passed, 0 failed/errors, 0 skipped**.

Commands executed

- `bash scripts/backend.sh pytest --junitxml=../artifacts/backend-tests.xml`
- `cd dashboard && npm test -- --run --reporter=junit --outputFile=../artifacts/dashboard-tests.xml`
- `cd mobile && npm test -- --run --reporter=junit --outputFile=../artifacts/mobile-tests.xml`
- `cd dashboard && npx playwright test`

Build and runtime validation

Capability	Observed outcome
Execution environment	Linux x86_64; Python 3.12, Node 24, PostgreSQL 16; no macOS ARM64 execution.
Reusable cloud setup	PASS: bash scripts/cloud_install.sh completed; npm ci, uv sync --frozen, three checksum-verified models, migrations, repeatable seed and both Docker builds. Existing local secrets preserved.
Database schema	PASS: alembic upgrade head and alembic check; no new upgrade operations detected; PostgreSQL integration tests passed.
Backend image and runtime	PASS: docker build with trusted CA secret; docker compose up -d --no-build db backend dashboard recreated services successfully. GET /health reports database connected; GET /ready reports face_models loaded and attestation not_configured.
Dashboard production build	PASS: npm run build (TypeScript and Vite); nginx Docker image starts and serves the actual API-connected dashboard.
Browser integration	PASS: npx playwright test against running Docker services; administrator room/session/export/close workflow and scoped faculty workflow. Tests use the created session ID and wait for navigation to remain repeatable.
Mobile type checking	PASS: npm run typecheck after the verification-result navigation correction.
Mobile Android bundle	PASS: CI=1 EXPO_NO_TELEMETRY=1 npm run export after the navigation correction; Metro/Hermes produced the actual Android bundle.
Android native preparation	PASS: Expo Android prebuild and local spvs-wifi module autolinking. Java 17, Gradle 9.4.1, Android API/build tools 37 and NDK 27.1.12297006 installed for the attempted build.
Android APK build	BLOCKED: ./gradlew :app:assembleDebug --no-daemon --max-workers=2 -PreactNativeArchitectures=arm64-v8a failed while resolving react-android-0.88.0-debug.aar. Maven Central redirects to repo.reactnative.dev, blocked by the cloud proxy with HTTP 403. Domain added to the saved environment draft; apply configuration before retrying. No APK produced; native compilation remains unverified.
Physical phone, iOS and Apple Silicon	NOT RUN: requires device/macOS hardware. Camera liveness, actual GPS/Wi-Fi behavior, accessibility and latency need the documented device matrix.
Platform attestation	UNCONFIGURED: adapter returns unverified; external Play Integrity/App Attest developer configuration is required.
Cloud configuration persistence	SAVED DRAFT: tested install/start instructions plus repo.reactnative.dev network addition. Saving is not applying/publishing; fresh-task restoration has not been independently tested.

Dependency audits

- Backend: 0 runtime packages with reported vulnerabilities in the locked-requirements audit.

- Dashboard: 0 affected package entries (0 critical, 0 high, 0 moderate, 0 low).

- Mobile: 45 affected package entries (0 critical, 45 high, 0 moderate, 0 low).

Audit commands: `npm audit --json` in each client; `pip-audit -r .runtime/backend-requirements.txt --format json` using requirements exported from `backend/uv.lock`. Raw reports are in `docs/evidence`.

Interpretation and unverified behavior

The backend's four real-model smoke cases are included in its total, not counted twice. Its API transaction happy paths explicitly substitute vision inside pytest fixtures; they do not prove live human capture. Real-model smoke tests execute the current YuNet, SFace and MediaPipe Tasks pipeline. No physical phone, iOS device, macOS ARM64 run, provider-backed attestation or population biometric evaluation was performed.

The mobile audit's affected entries propagate two root advisories through the Expo/Metro tree: GHSA-vfj7-8cjw-p6xm (`braces`) and GHSA-86w9-cpqp-85rv (`node-forge`). The registry reported no fixed version. See `KNOWN_LIMITATIONS.md`; do not interpret build success as security certification.

No manual sensor or live-biometric success is claimed. Intermediate failures included a missing graphics runtime, cloud CA trust configuration, older dependency advisories and test locator/assertion errors; these were diagnosed before the recorded final successful checks. Any still-blocked native capability appears in the build table.

Requirements traceability

Requirement	Implementation	Automated evidence	Remaining validation
Persistent normalized storage	20 SQLAlchemy tables + explicit Alembic migration	Migration/table/constraint tests on PostgreSQL	Deployment backups and restore drill
Password/JWT/refresh security	`core/security.py`, `api/auth.py`	Login, invalid JWT, rotation, reuse and logout tests	Operational key rotation
Backend role restrictions	`roles`, `access_course`, `session_ids`	Student/staff denial, cross-course checks and role-change tests	Institution-specific account lifecycle
Academic management	`api/academic.py`, dashboard courses/classrooms/sessions	CRUD/session lifecycle and browser workflow	Real campus data entry
Actual face recognition	YuNet + SFace in `services/vision.py`	Real same-face/different-person/no-face/multi-face tests	Consented local calibration
Enrollment consent/deletion	`/biometrics/enroll`, DELETE `/biometrics`	API consent/storage/deletion contract tests	Live student enrollment
Temporal liveness	MediaPipe landmarks + sequence validator	EAR/pose sequence, timing, expiry tests	Physical phone spoofing/usability matrix
Device binding	Random key + HMAC + unique active-device index	Signature/device mismatch, second-device denial and reset tests	SecureStore on physical Android/iOS
Platform integrity	`services/attestation.py` fail-closed boundary	Unconfigured policy never passes as verified	Provider accounts, verifier, nonce and verdict tests
GPS evidence	Haversine + uncertainty/freshness gates	Boundary/stale/inaccurate/mock/travel tests	Indoor GPS field measurements
Wi-Fi evidence	Native Android scan module + reference matching	Matching/weak/unavailable policy tests	Kotlin build and real AP scans; see current report
Deterministic decision	`/attendance/verify` + presence service	Hard-gate and decision API tests	Full real-device happy path
Concurrency/idempotency	Transaction locks + unique constraints	Simultaneous identical requests, replay and duplicates	Load testing
Auditing/anomalies	Structured events and correction ledger	Repeated failures, device reset, policy/role/correction audit	External immutable sink if required
Faculty reports	Aggregates, Recharts and scoped CSV	CSV escaping, live browser export, actual API counts	Large data export performance
Student disputes	Mobile history + scoped dispute API	Ownership and resolution tests	Real student support process

Requirement	Implementation	Automated evidence	Remaining validation
Dashboard quality	React, router, query, labeled forms and responsive CSS	Interaction tests, production build, Chromium workflow	Assistive technology and usability study
Student app integration	Expo camera/location/Secure Store/API signing	TypeScript, HMAC/boundary tests, Metro/Hermes export	Physical sensor/permission tests
Documentation/artifacts	Markdown, diagrams, PDF/ZIP packaging script	PDF text/page and ZIP CRC/contents checks	Fill academic identity placeholders

“Automated” in this table does not imply manually tested. The generated verification report identifies the current counts and commands. No fake biometric approval or sensor measurement is used by application code.

Known limitations and release boundary

Verification assurance

- A normal phone can be rooted, instrumented or used with injected camera/location data. The HMAC proves possession of a credential, not trustworthy physical capture.

- Temporal landmark liveness is a heuristic. It is not ISO-certified presentation-attack detection, and sophisticated replay or deepfake attacks may succeed. The action vocabulary is small even though challenge IDs and ordering are randomized.

- SFace is a real pretrained model, but the chosen threshold has not been calibrated on this institution. The public-portrait smoke tests do not estimate FAR, FRR, fairness or operational accuracy.

- Real-camera head-direction conventions, blink timing, enrollment usability and sustained phone-to-server attendance must be validated on physical devices. Automated transaction tests use an explicitly substituted vision component; they do not replace that evaluation.

- GPS and Wi-Fi support approximate physical presence, not absolute classroom proof. Rounded location history supports only coarse anomaly rules. The absence of a mock-location flag does not establish honesty.

- Wi-Fi scans are implemented only through a local Android module and depend on OS permissions, enabled location, recent scan availability and throttling. iOS reports unsupported. The default fallback is manual review.

- Play Integrity and App Attest are unconfigured provider boundaries. Their adapter always returns unverified. Requiring attestation moves otherwise eligible attempts to review.

Application and operational limits

- Web lists generally show the latest 500 entries. Large datasets need cursor pagination and indexed query tuning; exports can be large.

- Session-row locks serialize verification and manual correction within a session. This prioritizes consistency and requires throughput evaluation before large lectures.

- Inference is CPU-based and serialized per worker. Model memory is repeated across workers; no measured capacity or latency SLA is claimed.

- Dashboard access tokens live in memory. Reload requires login. Logout revokes refresh tokens; existing access tokens have a maximum 15-minute default expiry window unless the user is disabled.

- Enrollment response recovery uses `/biometrics` status; attendance has durable idempotency, while enrollment replay returns a conflict after success.

- Audit records are protected through application authorization, not an external tamper-proof log. A database administrator can modify storage.

- No automated retention purge, key rotation/re-encryption job, institution SSO, password reset/email service or external audit export sink is included. Identity/role provisioning is institution-managed in the dashboard.

- In a real college, biometric consent, an accessible alternative attendance process, retention and dispute policies require institutional decisions. This software does not supply those policies.

- Demo Compose uses a database-owner account, local development secrets and no TLS reverse proxy. Harden deployment according to `DEPLOYMENT_GUIDE.md` before using real student data.

Dependency audit boundary

The current backend and dashboard audit results are recorded in `TEST_RESULTS.md` and the verification report. The Expo dependency tree currently contains two upstream advisories without published fixes: `braces` nested-pattern stack exhaustion (GHSA-vfj7-8cjw-p6xm) and `node-forge` RSA signature verification (GHSA-86w9-cpqp-85rv). npm propagates these through 45 affected package entries; that is not 45 distinct root flaws. These concern Metro/build tooling and Expo certificate utilities in this project. The application's HMAC implementation uses `@noble/hashes` and the server uses Python cryptography. Do not expose the development bundler to untrusted networks or treat this explanation as a general security clearance. Re-audit before distribution and adopt upstream fixes when available.

See the verification report for exact build, native compilation and physical-device test status. An exported JavaScript bundle is not an installed/tested Android APK, and Linux validation is not macOS ARM64 validation.

Native build blocker observed in this cloud run

Android prebuild and native module autolinking succeeded. The Gradle debug build reached Expo Kotlin compilation but could not download React Native 0.88.0’s Android AAR: Maven Central redirects to `repo.reactnative.dev`, which the current cloud proxy blocks with HTTP 403. That domain was added to the saved environment draft; the draft must be applied before another native build can validate compilation. No APK or native Kotlin compilation success is claimed. The Android JavaScript/Hermes export passed separately.

References and provenance

The following are implementation references, not claims of deployment or experimentally measured accuracy. Consult the linked versioned source and official documentation when reproducing the project.

1. FastAPI documentation: https://fastapi.tiangolo.com/ — dependency injection, validation, security and API documentation.

2. SQLAlchemy 2.0 documentation: https://docs.sqlalchemy.org/en/20/ — ORM mappings, sessions and transactions.

3. PostgreSQL 16 documentation: https://www.postgresql.org/docs/16/ — constraints, indexes and row-level locking.

4. Alembic documentation: https://alembic.sqlalchemy.org/ — migration generation and application.

5. OpenCV Zoo, pinned commit: https://github.com/opencv/opencv_zoo/tree/48b10fa505aeb51fbd127fc60a5c660040e823b7 — official YuNet/SFace assets, Git LFS object hashes and licenses.

6. OpenCV DNN face tutorial: https://docs.opencv.org/4.x/d0/dd4/tutorial_dnn_face.html — detection, aligned SFace features and matching metrics. Its example threshold is not a local SPVS calibration result.

7. MediaPipe Face Landmarker guide: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker — Tasks API and official model bundle.

8. MediaPipe source/license: https://github.com/google-ai-edge/mediapipe — Apache 2.0 code and model-related resources.

9. Argon2-cffi documentation: https://argon2-cffi.readthedocs.io/ — maintained password hashing interface.

10. Cryptography Fernet documentation: https://cryptography.io/en/latest/fernet/ — authenticated symmetric encryption and key handling.

11. Expo documentation: https://docs.expo.dev/ — development builds, Camera, Location, SecureStore and local modules.

12. Android Wi-Fi scanning overview: https://developer.android.com/develop/connectivity/wifi/wifi-scan — scan restrictions, permissions and throttling.

13. Android Play Integrity overview: https://developer.android.com/google/play/integrity/overview — external provider integration requirements.

14. Apple App Attest: https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity — attestation boundary not configured in this distribution.

15. React, Vite and TanStack Query: https://react.dev/ ; https://vite.dev/ ; https://tanstack.com/query/latest — client architecture.

16. Playwright and pytest: https://playwright.dev/ ; https://docs.pytest.org/ — browser and backend testing.

17. scikit-image astronaut sample, version 0.24.0: https://github.com/scikit-image/scikit-image/blob/v0.24.0/skimage/data/astronaut.png — public NASA astronaut portrait used for model smoke tests.

18. matplotlib Grace Hopper sample, version 3.9.2: https://github.com/matplotlib/matplotlib/blob/v3.9.2/lib/matplotlib/mpl-data/sample_data/grace_hopper.jpg — public reference portrait used for a different-person smoke test.

19. npm advisory GHSA-vfj7-8cjw-p6xm: https://github.com/advisories/GHSA-vfj7-8cjw-p6xm — unresolved `braces` issue recorded by audit.

20. npm advisory GHSA-86w9-cpqp-85rv: https://github.com/advisories/GHSA-86w9-cpqp-85rv — unresolved `node-forge` issue recorded by audit.

License texts for the downloaded face detector and recognizer are included in `LICENSE-YUNET.txt` and `LICENSE-SFACE.txt`. The project source inherits the repository's MIT license. Third-party dependency licenses remain their respective authors' licenses. Reference portraits are downloaded only for local tests and are excluded from submission archives.